PONTIFÍCIA UNIVERSIDADE CATÓLICA DO PARANÁ
ESCOLA POLITÉCNICA
BACHARELADO EM ENGENHARIA DE COMPUTAÇÃO

ANDRÉ FABRICIO WOZNIACK
RICARDO HESPANHOL PAMPLONA

AGENTE AUTÔNOMO PARA REMEDIAÇÃO VERIFICADA DE VULNERABILIDADES COM CONTEXTO DO PROJETO

PROJETO FÍSICO – 2º BIMESTRE

Orientador: Prof. Julio Cesar Nievola

Visto do Orientador

CURITIBA
2026

RESUMO

Ferramentas de análise estática de segurança (SAST), como o Semgrep, apontam vulnerabilidades no código-fonte, mas a correção continua manual. Produtos comerciais já geram correções com modelos de linguagem (LLMs), e estudos recentes mostram que muitas dessas correções compilam e preservam o comportamento do sistema, mas não eliminam a falha. Este projeto especifica um agente autônomo de remediação que recebe o relatório do SAST e, para cada achado, investiga o repositório com ferramentas de busca e leitura, consulta uma base de conhecimento especializada por classe de vulnerabilidade e uma memória das correções já aceitas no projeto, propõe um *patch* compatível com a arquitetura existente e o submete a uma verificação em camadas: compilação, testes existentes, reanálise por dois analisadores, teste de exploração que falha antes e passa depois da correção, preservação de comportamento e restrições de arquitetura. O resultado é entregue com o código, o raciocínio e um laudo de verificação, e só é aplicado após aprovação humana. O contexto do projeto é mantido por um índice atualizado incrementalmente. Nesta fase, o trabalho se concentra em projetos C#/.NET e Java. Como não há um conjunto público de vulnerabilidades em C# com testes de exploração, o trabalho também constrói esse conjunto; em Java, usa o Vul4J. A avaliação compara três configurações: LLM com contexto mínimo, LLM com contexto recuperado e agente completo.

Palavras-chave: Remediação de vulnerabilidades. Agentes de LLM. Análise estática de código. Verificação de *patches*. Contexto de projeto.

SUMÁRIO

1 INTRODUÇÃO 4
1.1 CONTEXTO E PROBLEMA 4
1.2 CONCEITOS BÁSICOS 5
1.3 TRABALHOS RELACIONADOS E LACUNA 5
1.4 PERGUNTA DE PESQUISA E HIPÓTESES 6
1.5 OBJETIVOS 7
1.5.1 OBJETIVO GERAL 7
1.5.2 OBJETIVOS ESPECÍFICOS 7
1.6 ESCOPO DO PROJETO 8
1.7 RECORTE DESTA FASE: .NET E JAVA 8
1.8 CONTRIBUIÇÕES ESPERADAS 9
1.9 ESTRUTURA DO DOCUMENTO 9
2 DETALHAMENTO DO PROJETO 10
2.1 VISÃO GERAL EM BLOCOS 10
2.2 ESTRUTURAS DE DADOS COMUNS 11
2.3 DETALHAMENTO DOS BLOCOS 11
2.3.1 B1 – INGESTÃO E TRIAGEM DE ALERTAS 11
2.3.2 B2 – AMBIENTE ISOLADO 12
2.3.3 B3 – ÍNDICE DO PROJETO 12
2.3.4 B4 – BASE DO ESPECIALISTA 13
2.3.5 B5 – MEMÓRIA DE CORREÇÕES 14
2.3.6 B6 – AGENTE DE REMEDIAÇÃO 15
2.3.7 B7 – VERIFICAÇÃO EM CAMADAS E LAUDO 16
2.3.8 B8 – RELATÓRIO E REVISÃO HUMANA 17
2.3.9 B9 – ORQUESTRADOR EXPERIMENTAL 18
2.4 INTERFACES ENTRE BLOCOS 18
2.5 TECNOLOGIAS, FERRAMENTAS E CONFIGURAÇÃO 19
2.6 CONJUNTO DE CASOS 19
2.7 CONFIGURAÇÕES COMPARADAS 21
3 CRONOGRAMA DO PROJETO 22
4 PROCEDIMENTOS DE TESTE E VALIDAÇÃO DO PROJETO 23

4.1 TESTES DE CAIXA PRETA (PONTO DE VISTA DO USUÁRIO) 23
4.2 TESTES DE CAIXA BRANCA (PONTO DE VISTA DO SISTEMA) 23
4.3 DESENHO EXPERIMENTAL E MÉTRICAS 24
4.4 CRITÉRIOS DE ACEITE 25
5 ANÁLISE DE RISCOS 27
6 CONCLUSÃO 29
REFERÊNCIAS 31

1 INTRODUÇÃO

1.1 CONTEXTO E PROBLEMA

Ferramentas de análise estática de segurança, conhecidas pela sigla SAST (*Static Application Security Testing*), examinam o código-fonte sem executá-lo, em busca de padrões que costumam originar vulnerabilidades. Um exemplo é o uso de um dado digitado pelo usuário diretamente na montagem de uma consulta ao banco de dados ou de um caminho de arquivo. Ferramentas como o Semgrep (Semgrep, 2026b) e o SonarQube são amplamente adotadas, mas apresentam falsos positivos e dificuldade de compreender o contexto semântico do código (TechScience; Iranmanesh *et al.*, 2025, 2025). O uso crescente de assistentes de IA na escrita de código aumenta o volume de código a inspecionar e reproduz padrões inseguros (Pearce *et al.*; Veracode, 2022, 2025).

O resultado de uma ferramenta SAST é um *alerta*: ele indica a regra violada, o arquivo e a linha, mas não entrega uma correção. Corrigir o alerta continua sendo um trabalho manual, que exige entender como o restante do sistema acessa dados, trata erros, valida entradas e organiza suas camadas. Uma correção genérica pode eliminar o alerta e, ainda assim, quebrar o sistema, violar a arquitetura ou não corrigir a falha de fato.

Esse risco está documentado. Ao avaliar 319 *patches* gerados por um LLM para 64 vulnerabilidades Java reais, Al-Maamari (2026) verificou que apenas 24,8% estavam totalmente corretos: os *patches* preservavam a funcionalidade com frequência muito maior do que eliminavam a vulnerabilidade, e, para a CWE-20, nenhum corrigiu a falha. Avaliações baseadas em *exploits* reais apontam na mesma direção (Wang *et al.*, 2025). Em resumo, gerar um *patch* plausível é fácil; garantir que ele corrige a falha sem quebrar o projeto não é.

Em conversas informais com desenvolvedores de uma empresa do setor automotivo, o esforço para tratar os alertas de análise estática foi apontado como uma dor recorrente. Essas observações motivam o problema, mas não fornecem dados para o trabalho: nenhum código, alerta ou repositório corporativo será utilizado.

Na defesa do Plano de Projeto, a banca questionou o que a solução faria de

diferente em relação a ferramentas como SonarLint e SonarQube. A resposta adotada é **não competir com o mecanismo de detecção**: o trabalho consome os alertas de um analisador existente e se concentra em corrigi-los de forma compatível com o projeto e comprovada por verificação.

1.2 CONCEITOS BÁSICOS

– **CWE** (*Common Weakness Enumeration*): catálogo público que classifica tipos de fraqueza de *software*, como a CWE-89 (*SQL Injection*) e a CWE-22 (*Path Traversal*).

– **LLM** (*Large Language Model*): modelo de linguagem treinado em grandes volumes de texto e código, capaz de gerar código a partir de instruções.

– **Agente**: sistema em que o LLM não responde uma única vez, mas decide quais ferramentas usar (buscar, ler arquivos, editar, compilar, testar), observa os resultados e repete o ciclo até concluir a tarefa. O desenho das ferramentas oferecidas ao modelo influencia diretamente seu desempenho (Yang *et al.*, 2024).

– **RAG** (*Retrieval-Augmented Generation*): técnica em que um mecanismo de busca recupera documentos relevantes e os inclui na entrada do modelo, para que a resposta se apoie nessas informações (Lewis *et al.*, 2020). Aqui, os documentos são trechos do próprio repositório.

– ***Patch***: descrição das linhas a remover e a adicionar no código, no formato *unified diff* usado pelo Git.

– **Teste de exploração** (PoV, *proof of vulnerability*): teste automatizado que executa o ataque. Ele falha (a falha é explorável) no código original e passa no código corrigido.

– **Laudo de verificação**: registro de quais verificações a correção passou, entregue junto com o código.

1.3 TRABALHOS RELACIONADOS E LACUNA

Agentes de correção. O SWE-agent mostrou que um LLM com uma interface de ferramentas adequada resolve tarefas reais de manutenção de *software* (Yang *et al.*, 2024), e o RepairAgent aplicou essa ideia à correção de *bugs*, com o modelo

planejando e executando ações por meio de ferramentas (Bouzenia, Devanbu e Pradel, 2025). Para segurança, o PatchAgent combina localização da falha, geração e validação de *patches* usando um servidor de linguagem e um verificador, e relata a correção de mais de 90% de 178 vulnerabilidades reais (Yu *et al.*, 2025). O CodeRover-S adaptou um agente de correção às vulnerabilidades encontradas por *fuzzing* no OSS-Fuzz e corrigiu 61,1% de 588 casos históricos (Zhang *et al.*, 2026). O MemRepair acrescenta ao agente memórias de correções anteriores do repositório, de padrões de segurança e de tentativas que falharam antes de dar certo (Liu *et al.*, 2026).

Correção de alertas de análise estática. O PredicateFix usa os predicados das próprias regras do analisador para recuperar um exemplo de correção no código e orientar o LLM (Xiao *et al.*, 2026). Na indústria, o Copilot Autofix (GitHub, 2026b), o AI CodeFix da Sonar (SonarSource, 2026) e o Snyk Agent Fix (Snyk, 2026) geram correções com LLM para alertas de seus analisadores; são produtos fechados, sem avaliação pública e controlada.

Avaliação de correções. Métricas de semelhança com a correção humana superestimam o desempenho, e avaliações baseadas em testes revelam muitos *patches* incorretos (Han *et al.*, 2026). Por isso, trabalhos recentes avaliam por compilação, testes funcionais e testes de exploração (Al-Maamari; Wang *et al.*, 2026, 2025). Em *benchmarks* de CVEs reais, agentes gerais de engenharia de *software* corrigem no máximo 21% dos casos (Wang, Liu e Xiao, 2025), e uma sistematização recente aponta baixa eficácia das ferramentas em casos reais (Hu *et al.*, 2025).

Lacuna. Os agentes de correção de vulnerabilidades encontrados partem, em geral, de falhas encontradas por *fuzzing* ou de descrições de problemas, e não de alertas de análise estática; poucos combinam conhecimento especializado por classe de vulnerabilidade com o contexto do projeto, e são raros os que medem se a correção respeita a arquitetura existente. Este trabalho ocupa esse espaço: um agente que parte de alertas SAST, investiga o projeto, usa conhecimento especializado e comprova suas correções por verificação em camadas.

1.4 PERGUNTA DE PESQUISA E HIPÓTESES

Em que medida um agente autônomo que investiga o repositório, usa conhecimento especializado em vulnerabilidades e verifica suas próprias

correções produz correções de alertas SAST mais corretas e mais compatíveis com o projeto do que um LLM que gera a correção em uma única tentativa?

– **H1:** a proporção de correções totalmente corretas (compilam, preservam os testes, eliminam a falha comprovadamente e respeitam as restrições do projeto) é maior para o agente completo do que para o LLM em uma única tentativa, com ou sem contexto recuperado.

– **H2:** fornecer contexto do projeto aumenta a compatibilidade das correções com a arquitetura e as convenções existentes, mesmo sem o ciclo de verificação.

1.5 OBJETIVOS

1.5.1 OBJETIVO GERAL

Desenvolver e avaliar um agente autônomo de remediação de vulnerabilidades que recebe alertas de uma ferramenta SAST, investiga o repositório, propõe correções compatíveis com a arquitetura do projeto e comprova seus efeitos por meio de verificação em camadas, sob aprovação humana.

1.5.2 OBJETIVOS ESPECÍFICOS

a) Construir um índice do repositório, atualizado incrementalmente, que permita ao agente recuperar código, testes, configurações e documentação relacionados a cada alerta;

b) Elaborar uma base de conhecimento especializada, com fichas de correção por classe de vulnerabilidade e recomendações por linguagem e *framework*;

c) Implementar o agente, com ferramentas de investigação, edição e verificação, memória de correções do projeto e limites de iterações e custo;

d) Implementar a verificação em camadas e o laudo que acompanha cada correção;

e) Montar um conjunto de casos de vulnerabilidades com testes de exploração para as linguagens desta fase;

f) Comparar experimentalmente o agente com duas configurações de geração em uma única tentativa.

1.6 ESCOPO DO PROJETO

Faz parte do projeto:

– alertas de segurança do Semgrep, de qualquer regra, consumidos em JSON/SARIF; o agente classifica cada alerta e trata as classes cobertas pela base de conhecimento (Seção 2.3.4);

– contexto restrito ao repositório local, com índice atualizado incrementalmente;

– execução em cópia de trabalho isolada e contêiner sem rede; interface de linha de comando;

– aprovação humana obrigatória antes de qualquer alteração na *branch* principal; o código proposto é sempre exibido;

– as linguagens definidas no recorte desta fase (Seção 1.7).

Não faz parte do projeto:

– desenvolver um novo detector de vulnerabilidades ou reduzir o tempo de execução do SAST;

– vulnerabilidades em dependências de terceiros (análise de composição de *software*);

– *plugin* de IDE, interface web ou integração com o *pipeline* de integração contínua;

– aplicar correções sem aprovação humana;

– uso de código, alertas ou repositórios de empresas.

1.7 RECORTE DESTA FASE: .NET E JAVA

A abordagem do agente não depende de uma linguagem específica: o índice, as ferramentas, a memória e o ciclo de verificação funcionam sobre qualquer linguagem para a qual existam *parser*, regras do Semgrep, compilador e *framework* de testes. Para esta fase do projeto, o trabalho se concentra em duas linguagens:

– **C#/.NET:** os agentes de correção encontrados foram avaliados predominantemente em C/C++, Java e Python, e não foi encontrado um conjunto público de vulnerabilidades em C# com testes de exploração; os conjuntos multilinguagem que incluem C# não trazem esses testes (Bhandari, Naseer e Moonen; Wang *et al.*, 2021, 2023). A plataforma é amplamente usada em sistemas corporativos e é a de atuação profissional de um dos autores. Por essa lacuna, o trabalho constrói um conjunto de casos em C#.

– **Java:** conta com conjuntos públicos de vulnerabilidades reproduzíveis, como o Vul4J (Bui, Scandariato e Ferreyra, 2022), e com estudos recentes que usaram esse conjunto (Al-Maamari, 2026), o que permite comparar os resultados com a literatura.

1.8 CONTRIBUIÇÕES ESPERADAS

a) Um agente de remediação de código aberto cujas correções são acompanhadas de laudo de verificação;

b) Um conjunto de casos de vulnerabilidades em C# com testes de exploração e regras de arquitetura, reutilizável por outros trabalhos;

c) Uma avaliação controlada, em C# e Java, do efeito do contexto do projeto e do ciclo de verificação sobre a qualidade das correções.

1.9 ESTRUTURA DO DOCUMENTO

O Capítulo 2 apresenta o diagrama em blocos e o detalhamento de cada bloco, as interfaces, as tecnologias e o conjunto de casos. O Capítulo 3 traz o cronograma revisado. O Capítulo 4 descreve os testes de caixa preta e de caixa branca, o desenho experimental e os critérios de aceite. O Capítulo 5 apresenta a análise de riscos revisada, e o Capítulo 6, a conclusão.

2 DETALHAMENTO DO PROJETO

2.1 VISÃO GERAL EM BLOCOS

O sistema é inteiramente de *software* e é escrito em Python. Não há blocos de *hardware* nem de *firmware*; portanto, não há diagrama elétrico nem código em *Assembly*. Ele é executado por linha de comando em computadores de propósito geral (Linux ou macOS) com Docker e recebe dois insumos: um repositório Git de um projeto em uma das linguagens desta fase (C#/.NET ou Java, Seção 1.7) e o relatório de alertas do Semgrep. Para cada alerta tratável, entrega um *patch*, o raciocínio do agente e um laudo de verificação, que o desenvolvedor aprova ou rejeita.

A Figura 1 apresenta os blocos. A abordagem de descrição é a estruturada: para cada bloco são apresentadas entradas, saídas, processamento e pseudocódigo. As interfaces entre blocos estão resumidas na Tabela 3.

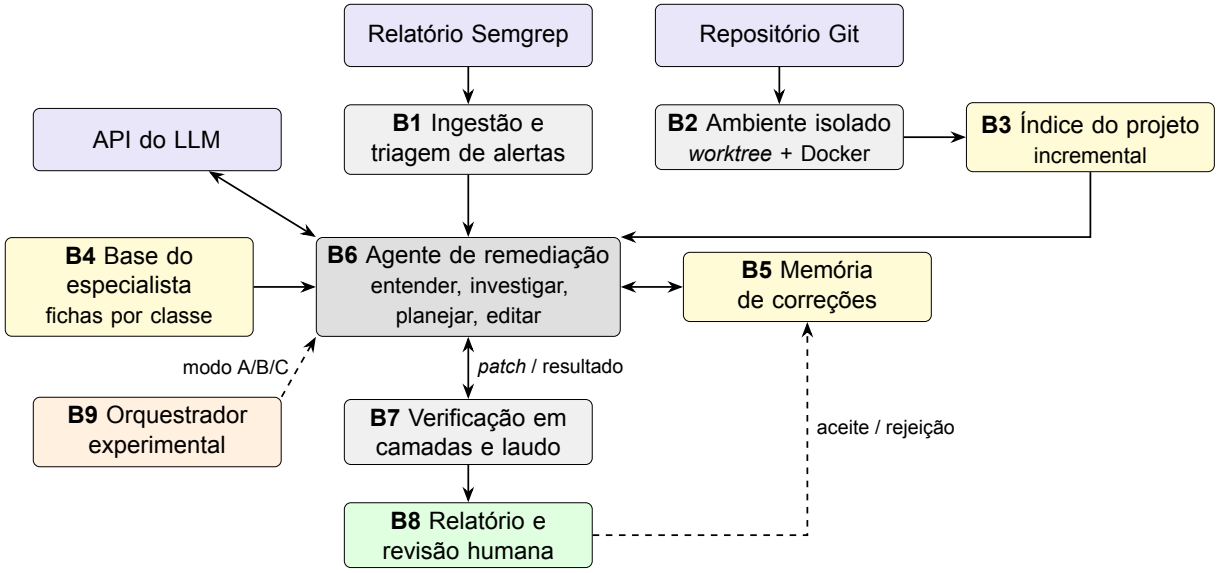


Figura 1 – Diagrama em blocos do agente de remediação.

Fonte: Elaborado pelos autores.

Os blocos em amarelo formam o conhecimento disponível ao agente: o índice do projeto (B3), a base do especialista (B4) e a memória de correções (B5). O agente (B6) é o único bloco que chama o LLM e usa o verificador (B7) dentro do seu ciclo. O orquestrador (B9) existe apenas para o experimento.

2.2 ESTRUTURAS DE DADOS COMUNS

Os blocos trocam registros serializáveis em JSON, o que permite testá-los isoladamente:

- `Alerta`: `id`, `regra`, `cwe`, `severidade`, `arquivo`, `linha_ini`, `linha_fim`, `mensagem`, `trecho`, `classe` (preenchida pela triagem);
- `Chunk`: `arquivo`, `simbolo`, `linha_ini`, `linha_fim`, `tipo` (código, teste, configuração, documentação), `hash`, `texto`;
- `Ficha`: classe de vulnerabilidade, CWEs, correção recomendada por *framework*, falsas correções, modelo de teste de exploração e verificações obrigatórias (Seção 2.3.4);
- `Memoria`: alerta, *patch*, decisão (aceito ou rejeitado), motivo, arquivos de origem e seus *hashes*;
- `Proposta`: `diff`, `plano` (causa e estratégia em texto), `evidencias` (trechos consultados), `iteracoes`, `tokens`, `custo`;
- `Laudo`: resultado de cada verificação (V1 a V8), `estado` final (verificado, parcial ou não corrigido) e tempos.

2.3 DETALHAMENTO DOS BLOCOS

2.3.1 B1 – INGESTÃO E TRIAGEM DE ALERTAS

Entrada: relatório do Semgrep em JSON ou SARIF. **Saída:** lista de `Alerta` classificados.

O bloco normaliza os resultados, remove duplicatas (mesmo arquivo, linha e regra) e atribui a cada alerta uma classe de vulnerabilidade a partir da CWE informada nos metadados da regra. Alertas cuja classe não tem ficha na base do especialista são marcados como *não tratáveis* e aparecem no relatório final, sem chamada ao LLM. O Semgrep é executado com as regras de segurança da comunidade para C# e Java, com versão fixada. A própria documentação do Semgrep informa que parte das regras de C# específicas de *frameworks* exige o motor comercial (Semgrep, 2026a); essa

limitação é registrada como risco, e o trabalho usa apenas o que a edição comunitária detecta.

```
funcao ingerir(relatorio, base):
    alertas = deduplicar(normalizar(relatorio.results))
    para cada a em alertas:
        a.classe = base.classe_da_cwe(a.cwe)     // None se nao houver ficha
        a.tratavel = (a.classe != None)
    retornar alertas
```

2.3.2 B2 – AMBIENTE ISOLADO

Cada alerta é tratado em uma cópia de trabalho própria (*git worktree*) criada a partir do *commit* analisado. Compilação, testes e análises rodam em um contêiner Docker sem acesso à rede, com limites de CPU, memória e tempo. As dependências (NuGet ou Maven) são baixadas antes, em uma etapa separada, e ficam em *cache*. Nenhum código gerado pelo agente é executado fora do contêiner, e a *branch* principal nunca é alterada por este bloco.

2.3.3 B3 – ÍNDICE DO PROJETO

Função. Permitir que o agente encontre o *contexto do projeto*: (a) implementações semelhantes e padrões já usados; (b) classes, interfaces e contratos relacionados; (c) bibliotecas, *frameworks* e versões (`.csproj`, `packages.config`, `pom.xml`); (d) convenções de arquitetura, tratamento de erros e validação; (e) testes existentes; e (f) documentação técnica.

Indexação. Os arquivos C# e Java são divididos em *chunks* por classe e por método com o *parser tree-sitter* (Tree-sitter, 2026), que possui gramáticas para as duas linguagens; arquivos de projeto, configuração (por exemplo, `appsettings.json`) e documentação em Markdown também são indexados. Cada *chunk* recebe uma entrada em um índice lexical BM25 e um *embedding* armazenado em uma base vetorial local (ChromaDB). Também é mantida uma tabela de símbolos (declarações e usos por nome), extraída pelo *tree-sitter*, usada para responder “quem chama” e “onde é usado”.

Modelo de *embedding*. O UniXcoder, considerado anteriormente, foi pré-treinado sobre linguagens que não incluem C# e, por isso, foi descartado. O modelo inicial é o `jina-embeddings-v2-base-code`, cuja documentação lista C# e Java entre as

linguagens suportadas, embora indique desempenho mais forte em outras linguagens (Jina AI, 2026). Por isso, a escolha é confirmada na primeira fase da implementação por uma prova de conceito: os casos anotados do conjunto C# (Seção 2.6) são usados para medir precisão@k e revocação@k do modelo, comparadas às da busca apenas lexical (BM25). Se o modelo não superar a busca lexical, o índice usa apenas BM25 e a tabela de símbolos. Os modelos da família CodeBERT (Feng *et al.*; Guo *et al.*, 2020, 2021) não são usados para gerar *patches*: sua janela de entrada curta não comporta o contexto do projeto.

Atualização incremental. Para cada arquivo guarda-se o *hash* SHA-256 do conteúdo e a versão do modelo e das regras de *chunking*. A cada execução, apenas arquivos novos ou alterados são reprocessados, e os removidos têm seus *chunks* apagados; se a versão do modelo ou do *chunking* mudar, o índice é reconstruído.

```
funcao atualizar_indice(repo, indice):
    se indice.versao != versao_atual(): retornar reconstruir(repo)
    atuais = {arq: sha256(arq) para arq em arquivos_indexaveis(repo)}
    para cada arq em indice.arquivos - atuais: remover(indice, arq)
    para cada arq, h em atuais:
        se indice.hash(arq) != h:
            remover(indice, arq)
            inserir(indice, chunkear(arq), embeddings(...), simbolos(arq), h)
```

2.3.4 B4 – BASE DO ESPECIALISTA

O “especialista” não é um modelo treinado à parte: é uma base de conhecimento curada, consultada pelo agente no início de cada alerta. Ela é organizada em fichas, uma por classe de vulnerabilidade, escritas em Markdown com cabeçalho estruturado. Cada ficha contém:

a) descrição da falha e condição de exploração;

b) correção recomendada por linguagem e *framework* (nesta fase, .NET e Java; por exemplo, parâmetros do ADO.NET, do Entity Framework e do JDBC para injeção de SQL);

c) falsas correções a evitar (por exemplo, escapar a *string* manualmente, usar lista de caracteres proibidos ou suprimir a regra com `nosemgrep`);

d) modelo de teste de exploração (xUnit em C#, JUnit em Java), ou a indicação de que a classe não admite esse teste (por exemplo, algoritmo criptográfico fraco);

e) verificações obrigatórias para a classe (Seção 2.3.7).

As fichas são redigidas a partir do OWASP Cheat Sheet Series (OWASP Foundation, 2026), do catálogo CWE (MITRE, 2026) e das descrições e referências das próprias regras do Semgrep. A versão inicial cobre as classes previstas no conjunto de casos (Tabela 1); outras classes podem ser acrescentadas sem alterar o código, apenas com novas fichas. A ideia de reutilizar padrões de defesa conhecidos corresponde à camada de padrões de segurança do MemRepair (Liu *et al.*, 2026).

Tabela 1 – Classes de vulnerabilidade cobertas pela versão inicial da base.

Classe	**CWE**	**Correção típica (exemplo em .NET)**
Injeção de SQL	CWE-89	consulta parametrizada (ADO.NET, EF)
Injeção de comando do SO	CWE-78	argumentos separados, sem *shell*; lista de permissão
Path traversal	CWE-22	normalizar caminho e validar a raiz permitida
Cross-site scripting	CWE-79	codificação de saída do *framework*
Desserialização insegura	CWE-502	desserializador seguro, tipos permitidos
XXE	CWE-611	desativar DTD e resolução externa
Criptografia fraca	CWE-327, CWE-328	algoritmo atual do `System.Security.Cryptography`
Credencial no código	CWE-798	leitura de configuração ou cofre de segredos
Redirecionamento aberto	CWE-601	validar destino local

Fonte: Elaborado pelos autores.

2.3.5 B5 – MEMÓRIA DE CORREÇÕES

Guarda as decisões do desenvolvedor sobre as propostas do agente no próprio projeto. Correções aceitas tornam-se exemplos de “como este projeto resolve esta classe de falha”; correções rejeitadas são guardadas com o motivo informado, para não serem repetidas. No início de cada alerta, o agente busca na memória as entradas da mesma classe e de arquivos relacionados. Cada entrada registra os *hashes* dos arquivos de onde veio; quando esses arquivos mudam, a entrada é marcada como desatualizada e deixa de ser oferecida como exemplo. A memória segue a ideia da camada de histórico de correções do MemRepair (Liu *et al.*, 2026). Nos experimentos,

a memória começa vazia em cada repositório, para não favorecer o agente com correções já validadas.

2.3.6 B6 – AGENTE DE REMEDIAÇÃO

Função. Para cada alerta tratável, produzir um *patch* que passe nas verificações obrigatórias. O agente segue um ciclo de ações e observações, com ferramentas definidas a seguir, à semelhança do SWE-agent e do RepairAgent (Yang *et al.*; Bouzenia, Devanbu e Pradel, 2024, 2025).

Ferramentas oferecidas ao LLM:

– `buscar(consulta)`: busca híbrida no índice B3;

– `simbolo(nome)`: declaração e usos de um símbolo;

– `grep(padrao)` e `ler(arquivo, linhas)`: busca textual e leitura;

– `estrutura()`: árvore de diretórios e projetos da *solution*;

– `ficha(classe)` e `memoria(classe, arquivo)`: consulta a B4 e B5;

– `editar(arquivo, trecho_antigo, trecho_novo)`: substituição exata na cópia de trabalho;

– `criar_teste(arquivo, conteudo)`: cria o teste de exploração em um projeto de testes separado, nunca nos testes existentes;

– `verificar()`: executa o bloco B7 e devolve o laudo parcial.

Ciclo. (1) *Entender:* ler o alerta, a ficha e o trecho. (2) *Investigar:* usar as ferramentas de busca para entender o fluxo do dado e como o projeto resolve problemas parecidos. (3) *Planejar:* registrar em texto a causa, a estratégia e por que ela segue o padrão do projeto. (4) *Escrever o teste de exploração*, quando a ficha o prevê, e confirmar que ele falha no código original. (5) *Editar* o código. (6) *Verificar.* Se alguma verificação obrigatória falhar, o resultado volta ao agente, que retoma a partir do passo 2.

```
funcao remediar(alerta, ferramentas, cfg):
    hist = [instrucoes_sistema(cfg), alerta, ferramentas.ficha(alerta.classe)]
```

```
para i em 1..cfg.max_iteracoes:              // padrao: 5
    enquanto nao estourou(cfg.max_acoes, cfg.max_custo):
        acao = llm(hist)                      // escolhe ferramenta ou conclui
        se acao.tipo == "concluir": sair
        hist += [acao, executar(acao)]        // observacao da ferramenta
    laudo = ferramentas.verificar()
    se laudo.obrigatorias_ok: retornar Proposta(diff(), plano(hist), laudo)
    hist += [laudo.falhas]
retornar NaoCorrigido(laudo, hist)
```

Restrições. As instruções proíbem e o bloco B7 verifica: alterar testes existentes, alterar configurações do Semgrep ou do CodeQL, inserir comentários de supressão, adicionar dependências não justificadas no plano e alterar arquivos sem relação com o alerta. **Limites:** no máximo 5 ciclos de verificação, 40 ações e um teto de custo por alerta, definidos no arquivo de configuração. Ao atingir um limite, o agente entrega “não corrigido” com o laudo da última tentativa. **Modelo:** Claude, via API, com modelo, versão e parâmetros registrados em cada execução.

2.3.7 B7 – VERIFICAÇÃO EM CAMADAS E LAUDO

A verificação segue a evidência da literatura de que compilar e eliminar o alerta não basta (Al-Maamari; Wang *et al.*, 2026, 2025). As verificações são executadas no contêiner, nesta ordem, interrompendo nas obrigatórias que falham:

Tabela 2 – Níveis de verificação do laudo.

Nível	Verificação	Como	Tipo
V1	Compila	`dotnet build` / `mvn compile`	obrigatória
V2	Testes existentes	mesmos resultados antes e depois (`dotnet test` / `mvn test`)	obrigatória
V3	Alerta resolvido	Semgrep não reporta o achado nem novos achados nas linhas alteradas	obrigatória
V4	Segunda análise	CodeQL não reporta a mesma CWE no trecho (GitHub, 2026a)	obrigatória se houver consulta para a CWE
V5	Exploração bloqueada	teste PoV falha no original e passa no corrigido	obrigatória se a ficha prever
V6	Comportamento preservado	testes gerados com entradas legítimas dão o mesmo resultado no original e no corrigido	obrigatória se V5 se aplicar
V7	Restrições e arquitetura	restrições do bloco B6 sobre o *diff* e testes de arquitetura do projeto (NetArchTest (Morris, 2026)), se existirem	obrigatória
V8	Revisão por LLM	segundo LLM, em papel de revisor de segurança, avalia o *diff* e o plano	informativa

Fonte: Elaborado pelos autores.

O laudo atribui um de três estados: **verificado** (todas as obrigatórias aplicáveis passaram, incluindo V5); **parcial** (as obrigatórias passaram, mas a classe não admite teste de exploração e a evidência de segurança vem apenas de V3 e V4); e **não corrigido**. O teste de exploração e os testes de comportamento são escritos pelo próprio agente; por isso, nos experimentos, cada caso também tem um teste de exploração de referência, escrito pela equipe e desconhecido do agente, usado na avaliação (Capítulo 4). O laudo nunca afirma que a correção é “garantida”: ele informa quais evidências foram obtidas.

2.3.8 B8 – RELATÓRIO E REVISÃO HUMANA

Para cada alerta, apresenta o *diff*, o plano do agente, os trechos consultados, o laudo e as limitações. O desenvolvedor sempre vê o código proposto antes de decidir. Comandos da CLI:

– `remediar indexar <repo>`: constrói ou atualiza o índice;

– `remediar corrigir <repo> <relatorio> [--modo A|B|C]`: trata os alertas;

– `remediar revisar <id> --aceitar | --rejeitar "motivo"`: registra a decisão na memória (B5); ao aceitar, cria uma *branch* com o *patch*, sem *merge*.

2.3.9 B9 – ORQUESTRADOR EXPERIMENTAL

Executa todos os casos em cada configuração (Seção 2.7) com os mesmos modelo, versão e parâmetros, aplica os testes de exploração de referência, registra tempo por etapa, *tokens*, custo e iterações em CSV e gera as tabelas de análise. Não faz parte do produto entregue ao desenvolvedor.

2.4 INTERFACES ENTRE BLOCOS

Tabela 3 – Interfaces entre os blocos.

De → Para	**Mecanismo**	**Dado**	**Formato**
Semgrep → B1	arquivo	relatório	JSON/SARIF do Semgrep
B1 → B6	chamada de função	`Alerta`	JSON
Repositório → B2	CLI do Git	*commit*	caminho da *worktree*
B2 → B3	sistema de arquivos	arquivos do projeto	caminhos e *hashes*
B3/B4/B5 → B6	ferramentas do agente	`Chunk`, `Ficha`, `Memoria`	JSON
B6 ↔ LLM	HTTPS	mensagens e chamadas de ferramenta	API do provedor
B6 ↔ B7	ferramenta `verificar`	cópia de trabalho; `Laudo`	JSON
B7 ↔ Docker	CLI do Docker	comandos de *build*, teste e análise	códigos de saída e saídas JSON/SARIF
B8 → B5	CLI	decisão e motivo	JSON
B9 → B6	configuração	modo A/B/C, limites	YAML

Fonte: Elaborado pelos autores.

2.5 TECNOLOGIAS, FERRAMENTAS E CONFIGURAÇÃO

Tabela 4 – Tecnologias selecionadas.

Bloco	**Tecnologia**	**Justificativa**
Todos	Python 3.12, *pytest*, Git	ecossistema de LLM e RAG; acessível a toda a equipe
B1, B7	Semgrep CLI (edição comunitária)	alertas de entrada; C# e Java suportados (Semgrep, 2026a)
B2, B7	Docker; .NET SDK 8; JDK 17 e Maven 3.9	isolamento e reprodutibilidade
B3	*tree-sitter* (C#, Java); *rank_bm25*; `jina-embeddings-v2-base-code`; ChromaDB	índice local, sem custo de API
B4, B5	Markdown com cabeçalho YAML; SQLite	legível por humanos e versionável
B6	API do Claude com chamada de ferramentas	raciocínio em várias etapas
B7	CodeQL CLI; xUnit/JUnit; NetArchTest	segunda análise, testes e regras de arquitetura
B8	Typer (CLI); Markdown e JSON	uso manual e integração futura
B9	YAML, pandas, SciPy	execução e análise estatística

Fonte: Elaborado pelos autores.

Plataforma alvo: Linux e macOS com Docker. **Compilação e depuração:** o sistema é interpretado (Python) e depurado com o depurador do VS Code e com *logs* estruturados de cada ação do agente; o código dos casos é compilado dentro do contêiner pelo .NET SDK ou pelo Maven. **Software de projeto de circuito elétrico:** não aplicável. **Configuração de software existente:** versões do Semgrep, das regras, do CodeQL e das imagens Docker são fixadas; chaves de API são lidas de variáveis de ambiente e nunca armazenadas no repositório. O CodeQL cria a base de dados de cada caso dentro do contêiner, usando o mesmo comando de *build* da verificação V1, e é executado com as consultas de segurança padrão para C# e Java.

2.6 CONJUNTO DE CASOS

Cada caso contém: a versão vulnerável do projeto, um alerta do Semgrep reproduzível, uma correção de referência, um teste de exploração de referência (quando a classe admite) e o comando de *build* e testes. Um caso só é admitido se compila de forma reprodutível no contêiner e se o Semgrep reproduz o alerta. Os números abaixo são metas; os números finais serão os casos que atenderem aos critérios e serão reportados como tal.

C# (conjunto construído neste trabalho):

a) **Juliet C# 1.3** (National Institute of Standards and Technology, 2026): suíte sintética do NIST que abrange 105 CWEs. São selecionados casos das classes da Tabela 1 que o Semgrep detecta. Meta: 20 casos. Testam a variedade de classes, mas não a arquitetura, pois os casos são pequenos e isolados.

b) **Vulnerabilidades inseridas no dotnet/eShop** (Microsoft, 2026): aplicação de referência da Microsoft em ASP.NET Core, com arquitetura em camadas e testes. A equipe insere vulnerabilidades das classes da Tabela 1 em pontos realistas, escreve os testes de exploração em xUnit e regras de arquitetura com NetArchTest. A versão original é a correção de referência. Meta: 15 casos. Testam diretamente a hipótese H2.

c) **Casos reais**: *commits* de correção em projetos C# obtidos do CVEfixes e do REEF (Bhandari, Naseer e Moonen; Wang *et al.*, 2021, 2023), filtrados manualmente pelos critérios de admissão. Meta: 5 casos, dependente da disponibilidade.

Java: subconjunto reproduzível do Vul4J (Bui, Scandariato e Ferreyra, 2022), que reúne 79 vulnerabilidades reais com correção humana e testes de prova de vulnerabilidade. O próprio repositório do Vul4J informa que parte dos casos deixou de ser reproduzível; por isso, só entram os casos que compilarem. Meta: 20 casos. Esse subconjunto permite comparar os resultados com o estudo de Al-Maamari (2026), que usou o mesmo conjunto.

Como os casos do Juliet, do Vul4J e de CVEs públicas podem ter sido vistos pelo LLM no treinamento, os resultados são reportados separadamente por origem, e os casos inseridos no eShop, criados pela equipe, servem de controle para essa ameaça.

2.7 CONFIGURAÇÕES COMPARADAS

Tabela 5 – Configurações do experimento.

Configuração	**Funcionamento**	**O que permite avaliar**
A – mínimo	uma chamada ao LLM com o alerta e o trecho vulnerável	linha de base
B – contexto	uma chamada com o alerta, a ficha do especialista e os trechos recuperados de B3, com orçamento fixo de *tokens*	efeito do contexto (H2)
C – agente	agente completo (B6) com ferramentas, memória e verificação em ciclo	efeito do agente e da verificação (H1)

Fonte: Elaborado pelos autores.

As três configurações usam o mesmo modelo, versão e parâmetros. As correções de A e B passam pelo bloco B7 apenas para avaliação, sem retorno ao modelo.

3 CRONOGRAMA DO PROJETO

O cronograma foi revisado após a defesa do Plano de Projeto e a mudança de ênfase, de detecção com *Machine Learning* para remediação com um agente de LLM. As Etapas 1 a 3 foram cumpridas: Formulário e Proposta, Plano de Projeto (entregue em 25/09/2026) e sua defesa (28/09/2026). A Etapa 4 (este Projeto Físico) encerra-se com a entrega em 12/11/2026. A partir da Etapa 6, a implementação segue a ordem de menor risco: primeiro o ambiente, as fichas do especialista e o conjunto de casos, junto com a verificação (B7), que é necessária para avaliar qualquer configuração; depois as configurações A e B; por último o agente completo (C). Assim, há resultados comparáveis mesmo que o agente atrase.

Na Figura 2, barras verdes indicam etapas cumpridas e barras azuis, etapas planejadas.

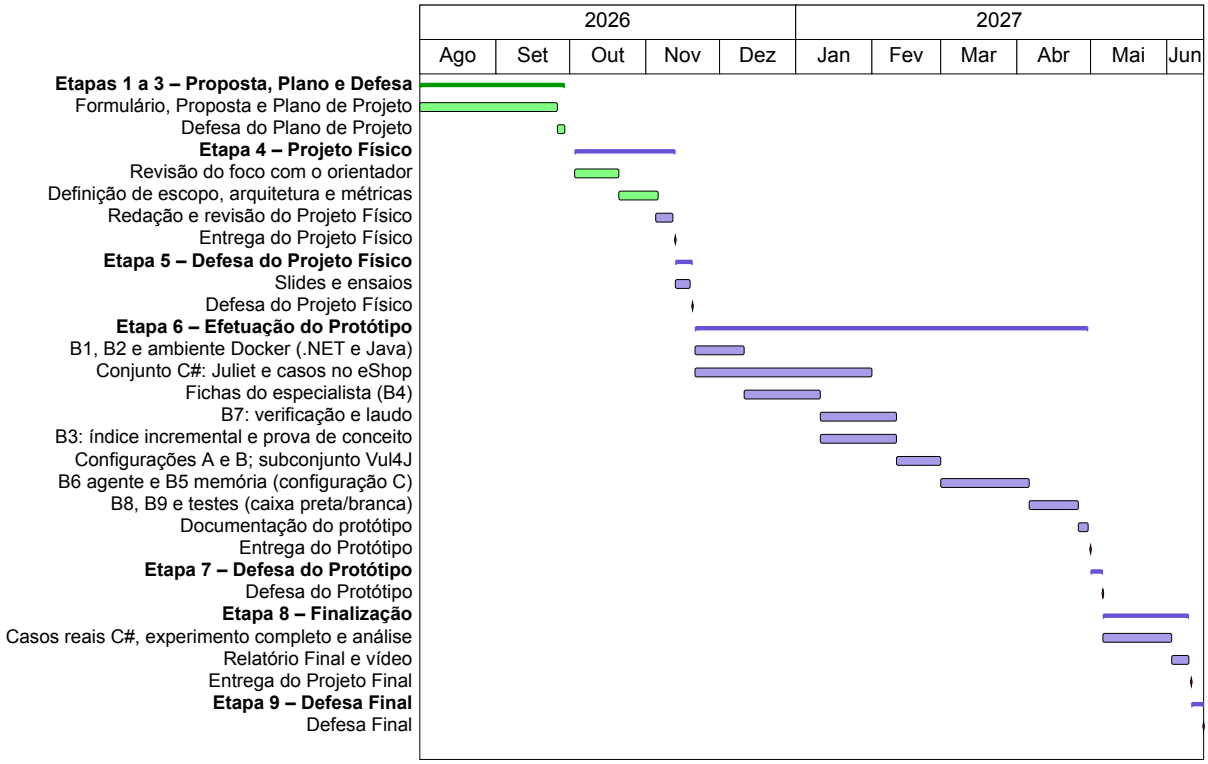


Figura 2 – Cronograma revisado do projeto – Gráfico de Gantt.

Fonte: Elaborado pelos autores.

4 PROCEDIMENTOS DE TESTE E VALIDAÇÃO DO PROJETO

4.1 TESTES DE CAIXA PRETA (PONTO DE VISTA DO USUÁRIO)

O usuário fornece um repositório e um relatório do Semgrep pela CLI e observa o relatório, o *diff* e o laudo. Os cenários usam projetos .NET de exemplo mantidos junto ao sistema.

Tabela 6 – Casos de teste de caixa preta.

ID	Entrada	Resultado esperado
CP-01	Projeto .NET com injeção de SQL e relatório válido	proposta com *diff*, plano, evidências e laudo no estado “verificado” ou “não corrigido”
CP-02	Alerta de classe sem ficha	alerta listado como não tratável; nenhuma chamada ao LLM
CP-03	Relatório malformado ou inexistente	erro claro, código de saída diferente de zero, nenhum arquivo alterado
CP-04	Alerta de criptografia fraca	laudo “parcial”, indicando que não há teste de exploração aplicável
CP-05	Caso em que o agente atinge o limite de ciclos	entrega “não corrigido” com o laudo da última tentativa, dentro do teto de custo
CP-06	`revisar --rejeitar` com motivo	motivo gravado na memória e exibido em alerta semelhante posterior
CP-07	`revisar --aceitar`	*branch* criada com o *patch*; *branch* principal inalterada
CP-08	Arquivo de origem de uma memória alterado	entrada da memória marcada como desatualizada

Fonte: Elaborado pelos autores.

4.2 TESTES DE CAIXA BRANCA (PONTO DE VISTA DO SISTEMA)

Cada bloco é testado isoladamente com *pytest*. O LLM é substituído por um simulador determinístico que devolve sequências de ações predefinidas, para que os testes sejam reprodutíveis e sem custo.

– **B1:** relatórios válidos, duplicados, sem CWE e malformados; classificação correta de CWEs em classes.

– **B2:** criação e remoção de *worktrees*; *hash* do repositório original igual antes e depois; contêiner sem acesso à rede.

– **B3:** *chunking* por classe e método em arquivos C# e Java de exemplo; tabela de símbolos com declarações e usos esperados; índice incremental idêntico

ao reconstruído do zero após criar, alterar e remover arquivos; precisão@k e revocação@k sobre casos anotados, comparando *embedding* e BM25.

– **B4/B5:** carga e validação das fichas; busca na memória por classe e arquivo; invalidação por mudança de *hash*.

– **B6:** com o simulador: execução de cada ferramenta; respeito aos limites de ciclos, ações e custo; retorno das falhas do laudo ao histórico; recusa de edições em testes existentes.

– **B7:** *patches* preparados para cada resultado esperado: correção válida (todas passam); *patch* que não compila (falha em V1); *patch* que quebra um teste (V2); *patch* que só suprime a regra (falha em V7); *patch* que engana a regra do Semgrep mas não corrige (passa em V3 e falha em V5).

– **Integração:** execução de ponta a ponta B1 a B8 sobre um caso do eShop com o LLM real.

4.3 DESENHO EXPERIMENTAL E MÉTRICAS

Cada caso do conjunto (Seção 2.6) é executado nas configurações A, B e C (Tabela 5), três vezes cada, para medir a variação do modelo. A avaliação usa os testes de exploração de referência da equipe, que o agente não vê.

– **Correção completa (métrica principal):** compila, mantém os testes existentes, o teste de exploração de referência passa a bloquear o ataque, o Semgrep não reporta o achado e as restrições de V7 são respeitadas. A mesma decomposição em compilação, segurança e funcionalidade é usada por Al-Maamari (2026), o que permite comparar os resultados do subconjunto Java;

– **Taxa por nível:** proporção de *patches* que passam em cada verificação V1 a V7, para localizar onde as configurações falham;

– **Falsa correção:** *patches* que passam em V3 (alerta sumiu) mas falham no teste de exploração de referência;

– **Concordância do laudo:** quantas correções marcadas como “verificado” pelo agente também passam no teste de referência;

– **Compatibilidade com o projeto:** nos casos do eShop, testes de arquitetura (NetArchTest) e uma rubrica de 0 a 2 em três itens (usa APIs e padrões já existentes, segue convenções de erro e validação, mantém o escopo mínimo), atribuída por dois avaliadores, com concordância medida pelo Kappa de Cohen;

– **Esforço:** iterações, ações, *tokens*, custo e tempo por etapa (indexação completa e incremental, investigação, geração, verificação);

– **Recuperação:** precisão@k e revocação@k do índice nos casos anotados.

Como as configurações são aplicadas aos mesmos casos, a comparação é pareada: teste exato de McNemar para as métricas binárias e teste de Wilcoxon para a rubrica. Com o número de casos previsto, as conclusões são apresentadas com a ressalva de baixo poder estatístico. Métricas de semelhança textual com a correção humana não são usadas como critério de sucesso, pois superestimam o desempenho (Han *et al.*, 2026).

4.4 CRITÉRIOS DE ACEITE

Tabela 7 – Critérios de aceite da solução.

Aspecto	**Critério de aceite**
Segurança da execução	Nenhum código gerado é executado fora do contêiner; a *branch* principal nunca é alterada sem `revisar --aceitar`.
Ingestão e triagem	Todos os alertas de relatórios válidos são normalizados e classificados; os não tratáveis são listados.
Índice	Índice incremental idêntico ao reconstruído; relatório de precisão@k e revocação@k disponível.
Agente	Respeita os limites configurados em todos os casos; toda proposta inclui *diff*, plano e evidências.
Verificação	Os *patches* preparados dos testes de caixa branca de B7 são classificados corretamente em 100% dos casos.

Aspecto	**Critério de aceite**
Conjunto de casos	Cada caso admitido compila no contêiner, reproduz o alerta e, quando aplicável, tem teste de exploração que falha na versão vulnerável e passa na de referência.
Experimento	Todos os casos executados nas três configurações com configuração idêntica; tabelas geradas automaticamente.
Reprodutibilidade	Modelo, versões, instruções, imagens Docker, regras e *commits* dos casos registrados; o experimento é reexecutável a partir de um arquivo de configuração.

Não se fixa uma meta de taxa de correção: o objetivo do experimento é medir a diferença entre as configurações, e uma meta anterior aos dados seria arbitrária.

5 ANÁLISE DE RISCOS

A análise do Plano de Projeto foi revisada. Riscos ligados à geração de grafos (CPG), ao treinamento de classificadores e ao desbalanceamento de dados de treinamento deixaram de existir com a mudança de foco. Surgiram riscos ligados ao comportamento do agente, à construção do conjunto de casos em C#, à cobertura do Semgrep para C# e à execução de código gerado. Probabilidade (P) e impacto (I) seguem a escala baixa, média e alta; o grau de exposição combina os dois (baixo, moderado, alto, crítico).

Tabela 8 – Análise de riscos revisada.

Risco	**P**	**I**	**Expos.**	**Redução**	**Contingência**
Prazo insuficiente para agente, conjunto de casos e experimento	Alta	Alto	Crítico	Ordem por menor risco: conjunto de casos e configuração A primeiro, agente depois	Reduzir o número de casos antes de cortar blocos
Construção do conjunto C# mais lenta que o previsto (casos que compilam, testes de exploração)	Alta	Alto	Crítico	Começar pelo eShop e pelo Juliet; modelo de teste de exploração por classe	Reduzir casos reais; priorizar casos inseridos
Semgrep comunitário detecta poucas classes em C#	Média	Alto	Alto	Verificar a cobertura das regras no início da Etapa 6	Restringir as classes às detectadas; registrar como limitação
Agente “engana” a verificação (altera testes, suprime regra, contorna o padrão)	Média	Alto	Alto	Restrições verificadas em V7; segunda análise (V4); teste de exploração de referência oculto	Contar como falha; reforçar as restrições

Risco	P	I	Expos.	Redução	Contingência
Custo da API ou ciclos longos do agente	Média	Médio	Moderado	Limites de ciclos, ações e custo; estimativa de custo por caso antes do experimento	Reduzir repetições ou usar modelo menor nas configurações A e B
Não determinismo do LLM	Alta	Médio	Alto	Três execuções por caso; registro de modelo, versão e parâmetros	Reportar a variação entre execuções
Contaminação: casos públicos vistos no treinamento do modelo	Média	Médio	Moderado	Casos inseridos pela equipe como controle; resultados por origem	Declarar como ameaça à validade
Embedding com desempenho fraco em C#	Média	Baixo	Baixo	Prova de conceito com precisão@k no início	Usar só BM25 e tabela de símbolos
Execução de código gerado causar dano ou vazar dados	Baixa	Alto	Moderado	Contêiner sem rede, limites de recursos, *worktree* descartável	Interromper e recriar o ambiente
Diferença de experiência em C# na equipe	Alta	Médio	Alto	Divisão de tarefas: núcleo .NET e casos C# com André; índice, agente em Python, Java e análise com Ricardo	Revisão de código cruzada; pareamento nas partes .NET
Uso de dados corporativos sem autorização	Baixa	Alto	Moderado	Somente repositórios e conjuntos públicos	Descartar o material

Os riscos de exposição crítica ou alta são revistos ao final de cada fase do cronograma (Figura 2). Os riscos de segurança da execução e de uso de dados corporativos são mitigados por projeto, pelos requisitos do bloco B2 e pelo escopo.

6 CONCLUSÃO

Este documento especificou um agente autônomo de remediação de vulnerabilidades. O agente recebe os alertas do Semgrep, investiga o repositório com ferramentas de busca e leitura, consulta uma base especializada por classe de vulnerabilidade e a memória das correções aceitas no projeto, propõe um *patch* e o submete a uma verificação em camadas antes de entregá-lo, com o código e um laudo, para aprovação humana. Essa escolha responde ao questionamento da banca sobre o diferencial em relação ao SonarLint e ao SonarQube: a contribuição está na correção, não na detecção.

A literatura recente sustenta as duas decisões centrais. Agentes com ferramentas superam a geração em uma única tentativa em tarefas de correção (Yang *et al.*; Yu *et al.*; Zhang *et al.*, 2024, 2025, 2026), e avaliações baseadas em testes de exploração mostram que a maioria dos *patches* gerados por LLMs não corrige a falha, ainda que compile (Al-Maamari; Wang *et al.*, 2026, 2025). Por isso, o laudo informa evidências, e não garantias. A abordagem não depende de uma linguagem específica; nesta fase, o trabalho se concentra em C#/.NET e Java. A escolha de C# responde a uma lacuna, pois não foi encontrado um conjunto público de vulnerabilidades em C# com testes de exploração, e o trabalho construirá esse conjunto; Java permite comparar os resultados com a literatura.

O sistema é composto por nove blocos de *software* (B1 a B9), cujas entradas, saídas, algoritmos e interfaces foram definidos no Capítulo 2. É escrito em Python e usa Semgrep, CodeQL, *tree-sitter*, BM25, *embeddings* de código, Docker, .NET SDK, JDK e a API de um LLM. A estratégia de desenvolvimento começa pelo conjunto de casos e pela configuração mais simples, para que haja resultados mesmo se o agente completo atrasar.

Os principais riscos são o prazo, o esforço de construir os casos em C#, a cobertura do Semgrep comunitário para C# e a possibilidade de o agente contornar a verificação; cada um tem medidas de redução e contingência no Capítulo 5. Considerando a evolução dos riscos e a maior compreensão da equipe sobre o problema, o projeto é viável, com a ressalva de que o número de casos será ajustado

ao tempo disponível antes de qualquer corte de bloco.

Ficam como trabalhos futuros: integração ao *pipeline* de integração contínua e à IDE; outras linguagens; uso de analisadores adicionais como fonte de alertas; e um modo de análise local durante a edição, restrito às alterações recentes.

REFERÊNCIAS

AL-MAAMARI, A. **Why LLMs Fail: A Failure Analysis and Partial Success Measurement for Automated Security Patch Generation**. 2026. ArXiv:2603.10072. Disponível em: https://arxiv.org/abs/2603.10072. Acesso em: 6 nov. 2026.

BHANDARI, G.; NASEER, A.; MOONEN, L. CVEfixes: Automated collection of vulnerabilities and their fixes from open-source software. In: **Proceedings of the 17th International Conference on Predictive Models and Data Analytics in Software Engineering (PROMISE)**. [S.l.: s.n.], 2021.

BOUZENIA, I.; DEVANBU, P.; PRADEL, M. RepairAgent: An autonomous, LLM-based agent for program repair. In: **Proceedings of the 47th International Conference on Software Engineering (ICSE)**. [S.l.: s.n.], 2025.

BUI, Q.-C.; SCANDARIATO, R.; FERREYRA, N. E. D. Vul4J: A dataset of reproducible Java vulnerabilities geared towards the study of program repair techniques. In: **Proceedings of the 19th International Conference on Mining Software Repositories (MSR)**. [S.l.: s.n.], 2022.

FENG, Z.; GUO, D.; TANG, D.; DUAN, N.; FENG, X.; GONG, M.; SHOU, L.; QIN, B.; LIU, T.; JIANG, D.; ZHOU, M. Codebert: A pre-trained model for programming and natural languages. In: **Findings of the Association for Computational Linguistics: EMNLP 2020**. [S.l.: s.n.], 2020. p. 1536–1547.

GitHub. **CodeQL CLI**. 2026. GitHub Docs. Disponível em: https://docs.github.com/en/code-security/concepts/code-scanning/codeql/codeql-cli. Acesso em: 6 nov. 2026.

GitHub. **Copilot Autofix for code scanning**. 2026. GitHub Docs. Disponível em: https://docs.github.com/en/code-security/concepts/code-scanning/copilot-autofix-for-code-scanning. Acesso em: 6 nov. 2026.

GUO, D.; REN, S.; LU, S.; FENG, Z.; TANG, D.; LIU, S.; ZHOU, L.; DUAN, N.; SVYATKOVSKIY, A.; FU, S.; TUFANO, M.; DENG, S. K.; CLEMENT, C.; DRAIN, D.; SUNDARESAN, N.; YIN, J.; JIANG, D.; ZHOU, M. Graphcodebert: Pre-training code representations with data flow. In: **International Conference on Learning Representations (ICLR)**. [S.l.: s.n.], 2021.

HAN, W.; KWAK, Y.; YU, M.; KIM, K.; LEE, Y.; MOON, H.; PAEK, Y. Rethinking the capability of fine-tuned language models for automated vulnerability repair. In: **Proceedings of the 48th International Conference on Software Engineering (ICSE)**. [S.l.: s.n.], 2026. ArXiv:2512.22633.

HU, Y. *et al.* SoK: Automated vulnerability repair: Methods, tools, and assessments. In: **Proceedings of the 34th USENIX Security Symposium**. [S.l.: s.n.], 2025. p. 4421–4440.

IRANMANESH, M. *et al.* **ZeroFalse: Improving Precision in Static Analysis with LLMs**. 2025. ArXiv:2510.02534. Disponível em: https://arxiv.org/abs/2510.02534. Acesso em: 15 set. 2026.

Jina AI. **jina-embeddings-v2-base-code**. 2026. Hugging Face. Disponível em: https://huggingface.co/jinaai/jina-embeddings-v2-base-code. Acesso em: 6 nov. 2026.

LEWIS, P.; PEREZ, E.; PIKTUS, A.; PETRONI, F.; KARPUKHIN, V.; GOYAL, N.; KÜTTLER, H.; LEWIS, M.; YIH, W.-t.; ROCKTÄSCHEL, T.; RIEDEL, S.; KIELA, D. Retrieval-augmented generation for knowledge-intensive NLP tasks. In: **Advances in Neural Information Processing Systems (NeurIPS)**. [S.l.: s.n.], 2020.

LIU, S.; ZHANG, L.; LIU, F.; LIAN, X.; LIU, Y.; ZHU, Y. **MemRepair: Hierarchical Memory for Agentic Repository-Level Vulnerability Repair**. 2026. ArXiv:2605.17444. Disponível em: https://arxiv.org/abs/2605.17444. Acesso em: 6 nov. 2026.

Microsoft. **eShop Reference Application**. 2026. GitHub. Disponível em: https://github.com/dotnet/eShop. Acesso em: 6 nov. 2026.

MITRE. **CWE – Common Weakness Enumeration**. 2026. Disponível em: https://cwe.mitre.org/. Acesso em: 6 nov. 2026.

MORRIS, B. **NetArchTest: a fluent API for .NET that can enforce architectural rules in unit tests**. 2026. GitHub. Disponível em: https://github.com/BenMorris/NetArchTest. Acesso em: 6 nov. 2026.

National Institute of Standards and Technology. **Juliet C# 1.3 – SARD Test Suite 110**. 2026. Disponível em: https://samate.nist.gov/SARD/test-suites/110. Acesso em: 6 nov. 2026.

OWASP Foundation. **OWASP Cheat Sheet Series**. 2026. Disponível em: https://cheatsheetseries.owasp.org/. Acesso em: 6 nov. 2026.

PEARCE, H.; AHMAD, B.; TAN, B.; DOLAN-GAVITT, B.; KARRI, R. Asleep at the keyboard? assessing the security of github copilot's code contributions. In: **2022 IEEE Symposium on Security and Privacy (SP)**. [S.l.: s.n.], 2022. p. 754–768.

Semgrep. **C# support**. 2026. Semgrep Docs. Disponível em: https://semgrep.dev/docs/languages/csharp. Acesso em: 6 nov. 2026.

Semgrep. **Semgrep: repositório e documentação da CLI**. 2026. GitHub. Disponível em: https://github.com/semgrep/semgrep. Acesso em: 6 nov. 2026.

Snyk. **Fix code vulnerabilities automatically**. 2026. Snyk User Docs. Disponível em: https://docs.snyk.io/scan-with-snyk/snyk-code/manage-code-vulnerabilities/fix-code-vulnerabilities-automatically. Acesso em: 6 nov. 2026.

SonarSource. **AI CodeFix**. 2026. Documentação do SonarQube Cloud. Disponível em: https://docs.sonarsource.com/sonarqube-cloud/ai-capabilities/ai-codefix. Acesso em: 6 nov. 2026.

TechScience. Static analysis techniques for secure software: A systematic review. **Journal of Cyber Security**, v. 7, n. 1, 2025. Disponível em: https://www.techscience.com/JCS/v7n1/64045/html. Acesso em: 15 set. 2026.

Tree-sitter. **Tree-sitter: a parser generator tool and an incremental parsing library**. 2026. Disponível em: https://tree-sitter.github.io/tree-sitter/. Acesso em: 6 nov. 2026.

Veracode. **2025 GenAI Code Security Report**. 2025. Relatório técnico. Disponível em: https://www.veracode.com/resources/reports/2025-genai-code-security-report/. Acesso em: 15 set. 2026.

WANG, C.; LI, Z.; PENG, Y.; GAO, S.; CHEN, S.; WANG, S.; GAO, C.; LYU, M. R. REEF: A framework for collecting real-world vulnerabilities and fixes. In: **Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)**. [S.l.: s.n.], 2023.

WANG, P.; LIU, X.; XIAO, C. CVE-Bench: Benchmarking LLM-based software engineering agent's ability to repair real-world CVE vulnerabilities. In: **Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics (NAACL)**. [S.l.: s.n.], 2025. p. 4207–4224.

WANG, W.; MA, W.; HU, Q.; ZHANG, Y.; SUN, J.; WU, B.; LIU, Y.; XU, G.; JIANG, L. **VulnRepairEval: An Exploit-Based Evaluation Framework for Assessing Large Language Model Vulnerability Repair Capabilities**. 2025. ArXiv:2509.03331. Disponível em: https://arxiv.org/abs/2509.03331. Acesso em: 6 nov. 2026.

XIAO, Y.-A.; WANG, W.; LIU, D.; ZHOU, J.; CHENG, S.; XIONG, Y. PredicateFix: Repairing static analysis alerts with bridging predicates. In: **Proceedings of the 48th International Conference on Software Engineering (ICSE)**. [S.l.: s.n.], 2026. ArXiv:2503.12205.

YANG, J.; JIMENEZ, C. E.; WETTIG, A.; LIERET, K.; YAO, S.; NARASIMHAN, K.; PRESS, O. SWE-agent: Agent-computer interfaces enable automated software engineering. In: **Advances in Neural Information Processing Systems (NeurIPS)**. [S.l.: s.n.], 2024.

YU, Z. *et al.* PatchAgent: A practical program repair agent mimicking human expertise. In: **Proceedings of the 34th USENIX Security Symposium**. [S.l.: s.n.], 2025. p. 4381–4400.

ZHANG, Y. *et al.* Fixing security vulnerabilities with agentic AI in OSS-Fuzz. In: **Proceedings of the 48th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)**. [S.l.: s.n.], 2026. Versão preliminar: arXiv:2411.03346.